



RAPPORT DEVELOPPEMENT D'APPLICATIONS MOBILE

Projet : Application Mobile de Gestion de Rendez dans les cliniques.

Etudiant: MILLOGO Maré Augustin 1er J

L3/Génie Logiciel

Enseignant : M. GANSBEOGOI. INTRODUCTION

I. Introduction Generale

Dans un monde où la technologie est en plein essor, il y a la présence d'outils sophistiqués nous permettant de gérer facilement nos besoins, d'acquérir plus de temps sur la gestion.

C'est dans ce contexte que nous allons aborder notre projet qui permettra la gestion des Rendez vous dans les cliniques a travers un systeme simple.

II. OBJECTIF

Notre objectif en réalisant ce projet, c'est de faciliter la prise de rendez vous.

Ainsi, c'est de :

- Développer une application mobile complete et fonctionnelle
- Connecter l'application à une API REST S
- Utiliser une base de données relationnelle (MySQL)

III. TECHNOLOGIES UTILISÉES

	Mobile	Backend	Donnees
Couche Technologie	React native	Node/Express	MySQL

Notre environnement de travaux se résume a des Outils comme

- VS Code pour le développement ,
- Rest-Client pour les tests des API,
- Drawio pour la modélisation

IV. ANALYSE ET CONCEPTION

En analysant le projet et en vue de sa conception, et comme besoins fonctionnels misent en place, nous avons réalisés à pile les fonctionnalités demandées.

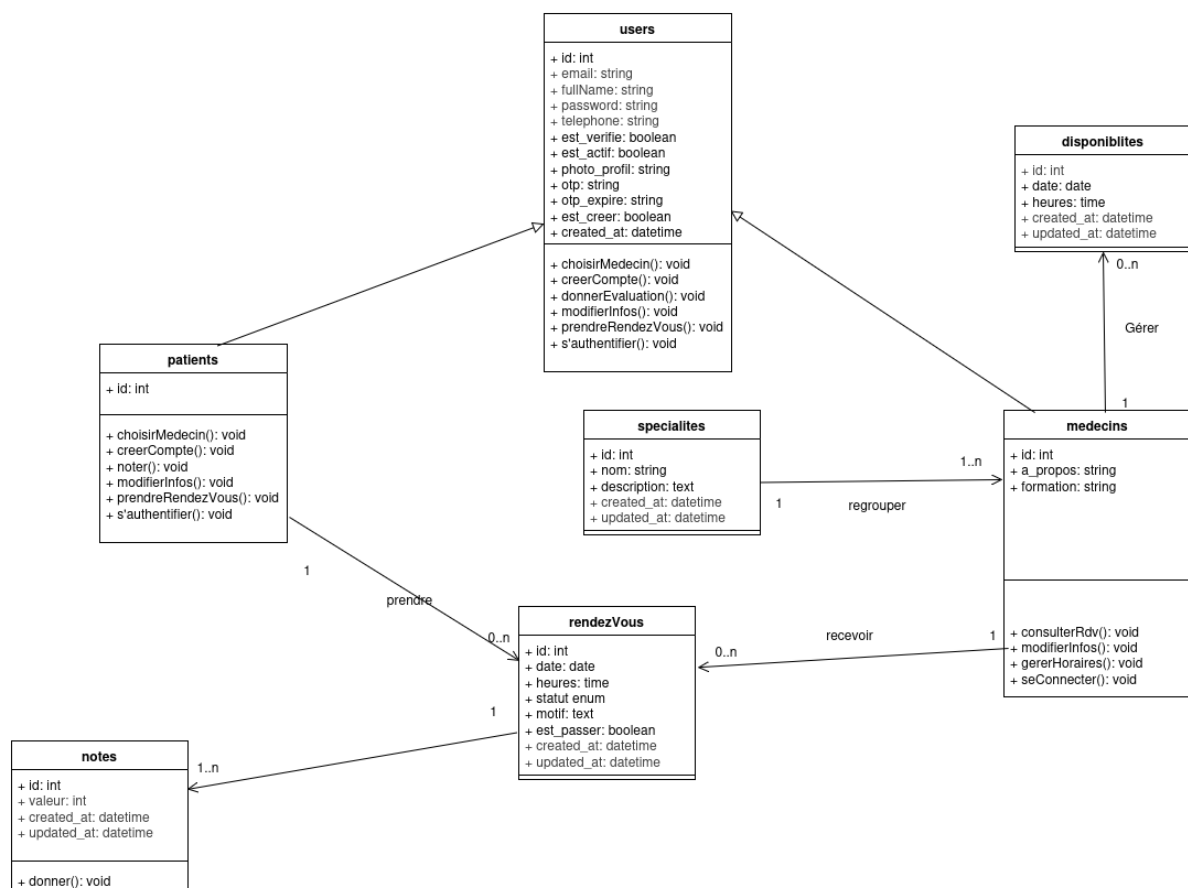
Sur lesquelles fonctionnalités, nous avons enumérés quelques une :

- Authentification
- Gestion des users
- Gestion de specialite
- Gestion de rendez vous

Et bien d'autres.

V.MODÉLISATION DES DONNÉES

Pour la modélisation.



Le MCD ci dessous:

MLD

users (id, email, fullName, telephone, password, photo_profil, otp, otp_expire, est_creer, est_verifie, est_actif)

patients (id, #userId)

admins (id, #userId)

medecins (id, a_propos, formation, #userId, #specialiteId)

specialites (id, nom, description)

disponibilites (id, date, heures, #medecinId)

rendezVous (id, date, heures, statut, motif, est_passer, #medecinId, #patientId)

notes (id, valeur, #rendezVousId)

Scripts SQL pour la cr base de données

-- Active: 1772416005858@@127.0.0.1@3306@AlloDoc /*** SCHEMA DB ALLO DOC **/

CREATE SCHEMA IF NOT EXISTS AlloDoc;

USE AlloDoc;

CREATE TABLE users (id INT AUTO_INCREMENT PRIMARY KEY NOT NULL, fullName VARCHAR(255) NOT NULL, email VARCHAR(50) UNIQUE NOT NULL, password VARCHAR(255) NOT NULL, photo_profil VARCHAR(255) NULL, telephone VARCHAR(50) UNIQUE NULL otp VARCHAR(255) NULL, otp_expire DATETIME NULL, est_actif BOOLEAN DEFAULT TRUE NOT NULL, est_verifie BOOLEAN DEFAULT FALSE NOT NULL, est_creer BOOLEAN DEFAULT FALSE NOT NULL, created_at DATETIME DEFAULT CURRENT_TIMESTAMP, updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP);

CREATE TABLE specialites (id INT AUTO_INCREMENT PRIMARY KEY NOT NULL, nom VARCHAR(50) UNIQUE NOT NULL, description TEXT NULL, created_at DATETIME DEFAULT CURRENT_TIMESTAMP, updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP);

```
CREATE TABLE patients ( id INT AUTO_INCREMENT PRIMARY KEY NOT NULL,
  userId INT NULL, created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
```

```
Foreign Key (userId) REFERENCES users(id) ON DELETE CASCADE
```

```
);
```

```
CREATE TABLE medecins ( id INT AUTO_INCREMENT PRIMARY KEY NOT NULL,
  a_propos TEXT NULL, formation TEXT NULL, userId INT NULL,
  specialiteId INT NULL, created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
```

```
Foreign Key (userId) REFERENCES users(id) ON DELETE CASCADE,
Foreign Key (specialiteId) REFERENCES specialites(id) ON DELETE CASCADE
```

```
);
```

```
CREATE TABLE admins ( id INT AUTO_INCREMENT PRIMARY KEY NOT NULL,
```

```
userId INT NULL,
```

```
Foreign Key (userId) REFERENCES users(id) ON DELETE CASCADE
```

```
);
```

```
CREATE TABLE disponibilites ( id INT AUTO_INCREMENT PRIMARY KEY NOT NULL,
  date DATE NOT NULL, heures TIME NOT NULL, created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
```

```
medecinId INT NULL,
```

```
Foreign Key (medecinId) REFERENCES medecins(id) ON DELETE CASCADE,
```

```
CONSTRAINT unique_creneau UNIQUE(medecinId, date, heures),  
  
);
```

```
CREATE TABLE rendezVous ( id INT AUTO_INCREMENT PRIMARY  
KEY NOT NULL, date DATE NOT NULL, heures TIME NOT NULL, statut  
ENUM('annulé', 'confirmé') DEFAULT 'confirmé' NOT NULL, motif TEXT  
NULL, est_passer BOOLEAN DEFAULT FALSE NOT NULL, medecinId INT  
NULL, patientId INT NULL, created_at DATETIME DEFAULT  
CURRENT_TIMESTAMP, updated_at DATETIME DEFAULT  
CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
```

```
Foreign Key (patientId) REFERENCES patients(id) ON DELETE CASCADE,  
Foreign Key (medecinId) REFERENCES medecins(id) ON DELETE  
CASCADE,  
CONSTRAINT unique_rdv UNIQUE(medecinId, patientId, date, heures),
```

```
);
```

```
CREATE TABLE notes ( id INT AUTO_INCREMENT PRIMARY KEY NOT  
NULL, valeur INT NOT NULL,
```

```
rendezVousId INT NULL,  
Foreign Key (rendezVousId) REFERENCES rendezVous(id) ON DELETE  
CASCADE
```

```
)
```

VI. TETS ET RÉSULTATS

Test réalisés au cours de la conception du projet :

- Test routes API avec REST-CLIENT
- Vérification des Résultats Obtenus :

- Application fonctionnelle
- Données persistées dans MySQL
- Synchronisation frontend – backend réussie

VII. CONCLUSION ET PERSPECTIVES

En réalisant ce projet, nous avons appris à gérer les données, et à réaliser une communication fluide entre un système de données en termes de persistance de données.